ISEN YNCRÉA OUEST

Majeure Cybersécurité - Sécurité des applications

Rapport de Travaux Pratiques

Sécurité Matérielle & Programmation Bas Niveau sur STM32

Assembleur Cortex-M4 • Exploitation et patch binaire • Protections mémoire Flash & SRAM (WRP, RDP, PCROP)

AUTEUR :

Félix MARQUET FISA 5

ENSEIGNANT :

Jean-Jacques MENEU

Année académique 2026-2027

Sommaire

- Introduction
- Construction d'un compteur modulo 10
- Conception d'une boucle for
- Création et appel d'une sous-routine
- Gestion du clignotement par interruption externe
- Localisation des variables en mémoire
- Écriture en mémoire Flash et RAM
  - Les adresses
  - Lecture de la Flash et de la RAM
  - Écriture en RAM
  - Écriture directe en Flash
  - Lecture à partir de 0x0802 0000
  - Programmation de la bank 2
  - Réécriture sans effacement
    - Remarque observée en pratique
  - Effacement de la bank 2
  - Nouvelle écriture
  - Synthèse
- Clignotement de LED et patch binaire
  - Le désassemblage
  - Le patch
  - La manipulation dans STM32CubeProgrammer
  - Résultat
- Protection en écriture mémoire (SRAM2 et Flash)
- Protection en lecture de la mémoire (RDP)
  - Niveau 0
  - Niveau 1
  - Régression du niveau 1 vers le niveau 0
- Protection propriétaire de code en exécution seule (PCROP)
  - État initial sans protection
  - Déplacement de la fonction dans le Linker Script
  - Configuration du compilateur (-mslow-flash-data)
  - Activation matérielle de la protection PCROP
  - Observation en mode Debug
  - Désactivation de la protection PCROP
- Isolation et enclave logicielle (Firewall)
  - Intégration manuelle du pilote HAL Firewall
  - Segmentation mémoire dans le Linker Script
  - Implémentation du code sécurisé et configuration
  - Analyse de l'implantation mémoire (Fichier .map)
  - Test 1 : Fonctionnement nominal et Pre-Arm
  - Test 2 : Tentative d'appel direct hors Call Gate
  - Test 3 : Tentative d'accès non autorisé aux données protégées
  - Test 4 : Analyse des fuites de données par les registres du cœur
    - Constat de la fuite d'information
    - Recommandations et contre-mesures

- Synthèse et bilan de sécurité

Introduction

```Unkown
maVariable:
     .word 2

main:
ldr r0, =maVariable // recupère l'adresse de la variable r0
ldr r1, [r0] // mets la valeur pointée par r0 dans r1

LoopForever:

     subs r1, #1 // soustrait 1 à r1 en mettant à jour le registre xPSR

     b LoopForever
```

Quand on ouvre la vue disasembly, on peut voir que `ldr r0, =maVariable` devient `ldr r0, [pc, #32] @ (0x800021c <LoopForever+30>)` à l'adresse `0x80001fa`. Puis quand on va voir dans le registre on peut voir que le PC (Program Counter) est en effet à la valeur `0x80001fa` ce qui veut dire que la prochaine instruction qui va être exécuté sera bien notre récupération d'adresse.
Une fois que l'on éxecute un step si on va à l'adresse correspondant à `r0` (`0x20000000`) on peut voir que la valeur est a 2 ce qui correspond a notre valeur initial, de plus le PC à changé pour `0x80001fc`.
Une fois que r1 est à 0, xpsr est à `0x61000000`, et une fois que r1 est à `0xffffffff`, xpsr est à `0x81000000` ce qui correspond à l'activation du flag *N* (Negative, bit 31), car le résultat de `0 - 1` est négatif, et du bit T* (Thumb, bit 24), qui est toujours à 1.

Construction d'un compteur modulo 10

```Unkown
count:
     .word 0

main:
ldr r0, =count

LoopForever:
     ldr r1, [r0] // r1 = count
     adds r1, #1 // On ajoute 1
     cmp r1, #10 // On regarde si r1 = 10
     blt Save // Si r1 < 10 on sauvegarde la valeur
     movs r1, #0 // Sinon on retourne a 0

Save:
     str r1, [r0] // count = r1
     b LoopForever
```

Les flags `xPSR` N Z C V correspondent respectivement à:

- **N (Negative, bit 31)** : vaut 1 si le résultat de l'opération est négatif
- **Z (Zero, bit 30)** : vaut 1 si le résultat de l'opération est nul
- **C (Carry, bit 29)** : vaut 1 s'il y a une retenue en sortie pour une addition, ou s'il n'y a **pas** d'emprunt pour une soustraction ou un `cmp`
- **V (oVerflow, bit 28)** : vaut 1 s'il y a un dépassement de capacité en arithmétique signée, c'est-à-dire quand le résultat ne tient pas dans 32 bits signés (par exemple, deux positifs dont la somme donne un négatif).

PC	Registre R0	Registre R1	xPSR (Flags N Z C V)	Count	Instruction	Remarque
0x80001fa	0x20000000	0x20000004	0x61000000 (N = 0, Z = 1, C = 1, V = 0)	0	`ldr r0, [pc, #40]`	État au démarrage

PC	Registre R0	Registre R1	xPSR (Flags N Z C V)	Count	Instruction	Remarque
0x80001fc	0x20000000	0x20000004	0x61000000 (N = 0, Z = 1, C = 1, V = 0)	0	`ldr r1, [r0, #0]`	R0 contient maintenant l'adresse de `count` : **0x20000000**
0x80001fe	0x20000000	0x0	0x61000000 (N = 0, Z = 1, C = 1, V = 0)	0	`adds r1, #1`	R1 = 0, valeur lue dans `count`
0x8000200	0x20000000	0x1	0x1000000 (N = 0, Z = 0, C = 0, V = 0)	0	`cmp r1, #10`	`adds` : résultat 1, positif, sans retenue, donc tous les flags à 0
0x8000202	0x20000000	0x1	0x81000000 (N = 1, Z = 0, C = 0, V = 0)	0	`blt.n 0x8000206`	`cmp 1, #10` donne 1 - 10 = -9 : N = 1, `blt` est pris car N ≠ V
0x8000206	0x20000000	0x1	0x81000000 (N = 1, Z = 0, C = 0, V = 0)	0	`str r1, [r0, #0]`	Saut vers `str` : `count` n'est pas encore mis à jour
0x8000208	0x20000000	0x1	0x81000000 (N = 1, Z = 0, C = 0, V = 0)	1	`b.n 0x80001fc`	`count` vaut 1 : écriture en mémoire effectuée, retour au début de la boucle
0x80001fc	0x20000000	0x1	0x81000000 (N = 1, Z = 0, C = 0, V = 0)	1	`ldr r1, [r0, #0]`	
0x80001fe	0x20000000	0x1	0x81000000 (N = 1, Z = 0, C = 0, V = 0)	1	`adds r1, #1`	
0x8000200	0x20000000	0x2	0x1000000 (N = 0, Z = 0, C = 0, V = 0)	1	`cmp r1, #10`	
0x8000202	0x20000000	0x2	0x81000000 (N = 1, Z = 0, C = 0, V = 0)	1	`blt.n 0x8000206`	
0x8000206	0x20000000	0x2	0x81000000 (N = 1, Z = 0, C = 0, V = 0)	1	`str r1, [r0, #0]`	
0x8000208	0x20000000	0x2	0x81000000 (N = 1, Z = 0, C = 0, V = 0)	2	`b.n 0x80001fc`	
0x80001fc	0x20000000	0x9	0x81000000 (N = 1, Z = 0, C = 0, V = 0)	9	`ldr r1, [r0, #0]`	
0x80001fe	0x20000000	0x9	0x81000000 (N = 1, Z = 0, C = 0, V = 0)	9	`adds r1, #1`	
0x8000200	0x20000000	0xa	0x1000000 (N = 0, Z = 0, C = 0, V = 0)	9	`cmp r1, #10`	
0x8000202	0x20000000	0xa	0x61000000 (N = 0, Z = 1, C = 1, V = 0)	9	`blt.n 0x8000206`	`cmp 10, #10` donne 0 : Z = 1, C = 1. `blt` **n'est pas pris** (N = V), donc PC passe à `0x8000204`
0x8000204	0x20000000	0xa	0x61000000 (N = 0, Z = 1, C = 1, V = 0)	9	`movs r1, #0`	`movs r1, #0` : R1 repasse à 0
0x8000206	0x20000000	0x0	0x61000000 (N = 0, Z = 1, C = 1, V = 0)	9	`str r1, [r0, #0]`	`str` sur le point d'écrire 0 dans `count`, qui vaut encore 9
0x8000208	0x20000000	0x0	0x61000000 (N = 0, Z = 1, C = 1, V = 0)	0	`b.n 0x80001fc`	`count` = 0 : le compteur a bouclé, 0 -> 1 -> ... -> 9 -> 0

Conception d'une boucle for

```
Unkown
sum:
    .byte 0

main:
ldr   r0, =sum          // r0 = adresse de sum

LoopForever:
    cmp   r1, #1            // compare X à 1
    blt   Start             // X < 1 (donc X = 0) : retour à sum = 0
    ldrb  r2, [r0]          // r2 = sum
    adds  r2, r2, r1        // sum = sum + X
    strb  r2, [r0]          // écrit sum (1 octet)
    subs  r1, r1, #1        // X = X - 1
    b     LoopForever

Start:
    movs  r2, #0
    strb  r2, [r0]          // sum = 0
    movs  r1, #20           // X = 20
```

Pour cette boucle on veut calculer la somme des 20 premiers entiers (1 à 20). Le résultat attendu est `(20 * 21) / 2 = 210 = 0xD2`, ce qui tient sur un seul octet. L'indice de la boucle `X` vit directement dans le registre `r1`. Au démarrage `ldr r0, =sum` place l'adresse de `sum` dans `r0`. Ensuite dans `Start` on initialise `sum = 0`. On utilise `strb` (et non pas `str` car `sum` ne fait qu'un octet et `str` écrit sur 4 octets et donc viendrait écraser les 3 octets voisins de `sum`).
Dans la boucle `cmp r1, #1` compare `X` à 1 et met à jour le `xPSR`. Si `X < 1` alors on passe dans `Start` qui comme dit plus tôt met `sum` à 0. Sinon on continue et on calcule `sum + X` avant de le stocker dans `r2`, une fois l'écriture en mémoire faite, on retire `1` à `X` et on recommence la boucle.

R1	Calcul	xPSR	blt
20 à 2	résultat positif	0x21000000 (C = 1)	non pris
1	1 - 1 = 0	0x61000000 (Z = 1, C = 1)	non pris
0	0 - 1 = -1	0x81000000 (N = 1, C = 0)	pris

Création et appel d'une sous-routine

```
Unkown
sum:
.byte 0

calculate_sum:
    movs r1, #0 // sum = 0
                // X = A : X est directement R0

LoopSum:
    cmp r0, #1 // compare X à 1
    blt EndSum // X < 1: fin du calcul
    adds r1, r1, r0 // sum = sum + X
    subs r0, r0, #1 // X = X - 1
    b LoopSum // retour au test

EndSum:
    mov r0, r1 // résultat renvoyé dans R0
    bx lr // retour à l'appelant

main:
ldr r2, =sum // r2 = adresse de la variable sum
```

```
Start:
    movs r0, #22 // A = 22
    bl calculate_sum // appel du sous-programme
    strb r0, [r2] // sum = R0
    b Start // on recommence
```

Pour ce programme on reprend le calcul de la somme, mais cette fois dans un sous-programme `calculate_sum` qui peut additionner n'importe quel nombre d'entiers. Ici on utilise `A = 22`, donc le résultat attendu est `22 * 23 / 2 = 253 = 0×FD`, ce qui tient encore sur un seul octet, `sum` reste donc en `.byte`.
Le sous-programme reçoit `A` dans `r0` et renvoie le résultat dans `r0`. `X` est directement `r0` et la somme est accumulée dans `r1`. Une fois la boucle finie, `mov r0, r1` place le résultat dans `r0`. On utilise `mov` sans le `s` car on n'a pas besoin de mettre à jour le `xPSR`. Le programme principal garde l'adresse de `sum` dans `r2`, que `calculate_sum` ne modifie pas, elle est donc encore valide après l'appel. Le résultat est ensuite écrit avec `strb`.
Durant l'exécution, au moment de rentrer dans le sous-programme, on peut voir que le `PC` passe de `0×800020e` à `0×80001fa` et que le `LR` passe de `0×80001fb` à `0×8000213`. Ce comportement vient de l'instruction `bl calculate_sum`, qui fait deux choses en même temps :

- Le `PC` reçoit l'adresse de la première instruction du sous-programme (`movs r1, #0`), c'est le saut.
- Le `LR` (Link Register) reçoit l'adresse de retour, c'est-à-dire l'adresse de l'instruction qui suit le `bl`. Le `bl` fait 4 octets en Thumb (instruction 32 bits), donc le `strb` est à `0×800020e + 4 = 0×8000212`. Le `LR` vaut `0×8000213` car le bit 0 est mis à 1 pour indiquer le mode Thumb.

Avant le `bl`, le `LR` valait `0×80001fb`, c'est l'adresse de retour du `bl main` fait dans `Reset_Handler`. Cette valeur est écrasée par le `bl calculate_sum`.
À la fin du sous-programme, `bx lr` recopie le `LR` dans le `PC`. Le processeur ignore le bit 0 et reprend à `0×8000212`, sur le `strb r0, [r2]`.

Moment	PC	LR
Avant `bl calculate_sum`	`0×800020e`	`0×80001fb`
Après `bl`	`0×80001fa`	`0×8000213`
Pendant la boucle	`LoopSum` à `EndSum`	`0×8000213`
Après `bx lr`	`0×8000212`	`0×8000213`

Gestion du clignotement par interruption externe

Unkown
```
init_led:
/* Configure PORT A*/

    /* Enable port A */
    ldr r0, =0×4002381C /* address of the RCC_AHBENR GPIO Clock Enable Register */
    ldr r1, =0×00000001 /* mask to apply to RCC_IOPENR */
    ldr r2, [r0] /* load value of RCC_IOPENR to r2 */
    orrs r1,r2 /* for IOPAEN to 1 to enable port A */
    str r1, [r0] /* write new value to RCC_IOPENR */

    /* configure pin in output mode */
    ldr r0, =0×40020000 /* address of the MODE Register port A */
    ldr r1, =0×00000400 /* mask to apply to MODE Register port A */
    ldr r2, [r0] /* load value of MODE Register port A */
    orrs r1,r2 /* force bit 10 to 1 */
    str r1, [r0] /* write new value to port A OTYPER */

    ldr r1, =0×FFFFF7FF /* mask to apply to MODE Register port A */
    ldr r2, [r0] /* load value of MODE Register port A */
```

```
    ands r1,r2 /* force bit 11 to 0 */
    str r1, [r0] /* write new value to port A OTYPER */

    bx lr
```

Le code fourni, dont voici un extrait ci-dessus, était écrit pour un STM32L152, alors que notre carte est un STM32L476. Au premier `ldr r2, [r0]` de `init_led`, le programme plantait.
Sur le L476, l'adresse `0x4002381C` (RCC_AHBENR du L152) se trouve dans une zone mémoire réservée. La lire provoque une BusFault, qui est escaladée en HardFault. Comme ce handler n'est pas défini, il pointe vers `Default_Handler`, et le programme reste bloqué dans `Infinite_Loop`.
Après analyse et recherche dans le reference manual du L476, voici les adresses à changer :

Registre	L152	L476
RCC, activation de l'horloge des GPIO	`0x4002381C` (RCC_AHBENR)	`0x4002104C` (RCC_AHB2ENR)
RCC, activation de l'horloge de SYSCFG	`0x40023820` (RCC_APB2ENR)	`0x40021060` (RCC_APB2ENR)
GPIOA_MODER	`0x40020000`	`0x48000000`
GPIOA_ODR	`0x40020014`	`0x48000014`
GPIOC_MODER	`0x40020800`	`0x48000800`

Cependant les bits à modifier restent les même donc seul les registres RCC et GPIO ont changé.

Localisation des variables en mémoire

Le STM32L476RG possède 1 Mo de Flash (à partir de `0x08000000`), 96 Ko de SRAM1 (à partir de `0x20000000`) et 32 Ko de SRAM2 (à partir de `0x10000000`). Le linker script de CubeIDE place le code et les constantes en Flash, et `.data`, `.bss` et la pile en SRAM1. La SRAM2 n'est pas utilisée ici.
Pour ce programme on déclare des variables dans les quatre zones demandées :

- **Flash** : `cst_int` et `cst_text` sont déclarées `const`, elles sont donc placées dans la section `.rodata`, en Flash. On s'attend à une adresse en `0x0800xxxx`.
- **SRAM, section `.data`** : `data_var1` et `data_var2` sont des variables globales initialisées avec une valeur non nulle. Elles sont en SRAM1, au début de la RAM (`0x2000xxxx`). Leur valeur initiale est stockée en Flash, puis copiée en RAM par `Reset_Handler` au démarrage.
- **SRAM, section `.bss`** : `bss_var1`, `bss_var2` sont des globales non initialisées. Elles sont en SRAM1, juste après `.data`, et mises à 0 par `Reset_Handler`.
- **Pile (stack)** : les variables locales `stack_var` (dans `function1`), `k`, `sw1` et `res` (dans `main`) sont des variables automatiques. `stack_var` et `k` sont `volatile`, donc forcées en mémoire. En `-O0`, `sw1`, `res` et `i` sont aussi en pile. La pile est en haut de la SRAM1 et descend depuis `_estack = 0x20018000`. Les adresses doivent donc être proches de `SP`, juste en dessous de `0x20018000`.
- **Registres** : `reg_var` est déclarée `register`, on s'attend à la voir dans un registre. Une variable en registre n'a pas d'adresse. En `-O1`, `sw1`, `i` et `res` passent aussi en registres.

Variable	Zone attendue	Adresse attendue	Adresse relevée
`cst_int`, `cst_text`	Flash (`.rodata`)	`0x0800xxxx`	`cst_int` : `0x8003090`, `cst_text` : `0x8003094`
`data_var1`, `data_var2`	SRAM1 (`.data`)	`0x2000xxxx` (début)	`data_var1` : `0x20000000`, `data_var2` : `0x20000004`
`bss_var1`, `bss_var2`	SRAM1 (`.bss`)	`0x2000xxxx` (après `.data`)	`bss_var1` : `0x200000b8`, `bss_var2` : `0x200000bc`
`stack_var`, `k`, `sw1`, `res`	Pile	juste sous `0x20018000`	`stack_var` : `0x20017fd8`, `k` : `0x20017fec`, `res` : `0x20017ff0`, `sw1` : `0x20017ff7`
`reg_var`	Registre	`R4` à `R11`	`R4`

&cst_text	const volatile char (*)[5]	0x8003094 <cst_text>
&data_var1	int *	0x20000000 <data_var1>
&data_var2	int *	0x20000004 <data_var2>
&bss_var1	int *	0x200000b8 <bss_var1>
&bss_var2	int *	0x200000bc <bss_var2>
&k	volatile int *	0x20017fec
&sw1	unsigned char *	0x20017ff7 "\001"
&res	int *	0x20017ff0

Écriture en mémoire Flash et RAM

Les adresses

Le STM32L476RG possède 1 Mo de Flash, 96 Ko de SRAM1 et 32 Ko de SRAM2.

Adresse	Zone
`0x0800 0000`	Début de la Flash (bank 1), qui contient la table des vecteurs.
`0x0808 0000`	Début de la bank 2 de la Flash (2 banks de 512 Ko en mode dual-bank).
`0x2000 0000`	Début de la SRAM1.

L'énoncé donne `0x8000 0000`, qui correspond en réalité à la zone FMC. Nous avons considéré qu'il s'agissait de `0x0800 0000`. De même, `0x08080 0000` et `0x8080 0004` sont lus comme `0x0808 0000` et `0x0808 0004`.

Lecture de la Flash et de la RAM

```c
volatile uint32_t val_flash, val_ram, readback, old, sr;

val_flash = *(volatile uint32_t *)0x08000000;
val_ram   = *(volatile uint32_t *)0x20000000;
```

- `val_flash` = `0x20018000` : c'est le premier mot de la table des vecteurs, la valeur initiale de `SP` (`_estack`).
- `val_ram` = `0x04C4B400` : c'est `SystemCoreClock`, la première variable de la section `.data`, mise à jour par `SystemClock_Config`.

Expression	Type	Value
val_flash	volatile uint32_t	0x20018000 (Hex)
val_ram	volatile uint32_t	0x4c4b400 (Hex)
readback	volatile uint32_t	0x80031d9 (Hex)
flash20000	volatile uint32_t	0x8003254 (Hex)
sr	volatile uint32_t	0x0 (Hex)
flash80004	volatile uint32_t	0x0 (Hex)
flash80000	volatile uint32_t	0x8003258 (Hex)
old	volatile uint32_t	0x80001ad (Hex)
FLASH->SR	volatile uint32_t	0x0 (Hex)
FLASH->CR	volatile uint32_t	0xc0000000 (Hex)

Écriture en RAM

```c
old = *(volatile uint32_t *)0x20000000;
*(volatile uint32_t *)0x20000000 = 0xDEADBEEF;
readback = *(volatile uint32_t *)0x20000000;
*(volatile uint32_t *)0x20000000 = old;
```

`readback` vaut `0xDEADBEEF` : la RAM s'écrit mot par mot, sans condition. On restaure ensuite l'ancienne valeur, car cette adresse contient `SystemCoreClock`.

Expression	Type	Value
(x)= val_flash	volatile uint32_t	0x20018000 (Hex)
(x)= val_ram	volatile uint32_t	0x4c4b400 (Hex)
(x)= readback	volatile uint32_t	0xdeadbeef (Hex)
(x)= flash20000	volatile uint32_t	0x8003254 (Hex)
(x)= sr	volatile uint32_t	0x0 (Hex)
(x)= flash80004	volatile uint32_t	0x0 (Hex)
(x)= flash80000	volatile uint32_t	0x8003258 (Hex)
(x)= old	volatile uint32_t	0x4c4b400 (Hex)
(x)= FLASH->SR	volatile uint32_t	0x0 (Hex)
(x)= FLASH->CR	volatile uint32_t	0xc0000000 (Hex)

Écriture directe en Flash

```c
*(volatile uint32_t *)0x08020000 = 0xDEADBEEF;
readback = *(volatile uint32_t *)0x08020000;
sr = FLASH->SR;
```

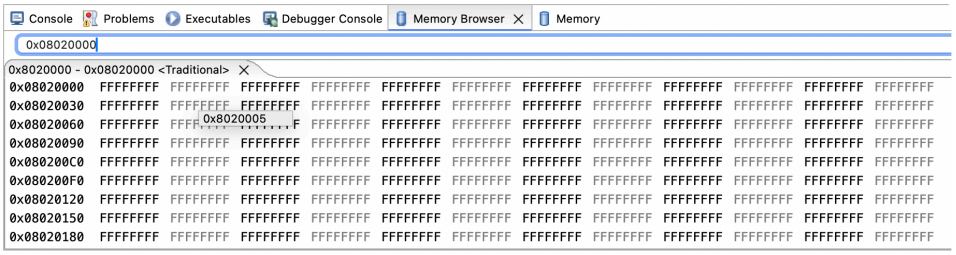
Chez nous, `readback` vaut `0xFFFFFFFF` : la valeur n'a pas été écrite. Il n'y a pas de HardFault, mais `FLASH->SR` = `0x80`, soit le flag **PGSERR**. La Flash est verrouillée (`FLASH->CR` = `0xC0000000`, bits LOCK et OPTLOCK) et il faut suivre une procédure précise pour la programmer, contrairement à la RAM.

Expression	Type	Value
(x)= val_flash	volatile uint32_t	0x20018000 (Hex)
(x)= val_ram	volatile uint32_t	0x4c4b400 (Hex)
(x)= readback	volatile uint32_t	0xffffffff (Hex)
(x)= flash20000	volatile uint32_t	0x8003254 (Hex)
(x)= sr	volatile uint32_t	0x80 (Hex)
(x)= flash80004	volatile uint32_t	0x0 (Hex)
(x)= flash80000	volatile uint32_t	0x8003258 (Hex)
(x)= old	volatile uint32_t	0x4c4b400 (Hex)
(x)= FLASH->SR	volatile uint32_t	0x0 (Hex)
(x)= FLASH->CR	volatile uint32_t	0xc0000000 (Hex)

Lecture à partir de `0x0802 0000`

La lecture donne `0xFFFFFFFF`. Oui, la mémoire est effacée à cet endroit : une cellule Flash effacée a tous ses bits à 1. Le programme fait environ 12 Ko et n'occupe donc pas cette zone.

Expression	Type	Value
(x)= val_flash	volatile uint32_t	0x20018000 (Hex)
(x)= val_ram	volatile uint32_t	0x4c4b400 (Hex)
(x)= readback	volatile uint32_t	0xffffffff (Hex)
(x)= flash20000	volatile uint32_t	0xffffffff (Hex)
(x)= sr	volatile uint32_t	0x80 (Hex)
(x)= flash80004	volatile uint32_t	0x0 (Hex)
(x)= flash80000	volatile uint32_t	0x8003258 (Hex)
(x)= old	volatile uint32_t	0x4c4b400 (Hex)
(x)= FLASH->SR	volatile uint32_t	0x0 (Hex)
(x)= FLASH->CR	volatile uint32_t	0xc0000000 (Hex)



Programmation de la bank 2

Procédure « Read from bank 1 while programming bank 2 » :

1. Déverrouiller la Flash en écrivant `0x45670123` puis `0xCDEF89AB` dans `FLASH_KEYR`. `FLASH→CR` passe de `0xC0000000` à `0x40000000`.
2. Attendre `BSY = 0` et effacer les flags d'erreur.
3. Mettre `PG = 1` dans `FLASH_CR`.
4. Écrire les deux mots : `0xDEADBEEF` à `0x08080000` et `0xAABBCCDD` à `0x08080004`.
5. Attendre `BSY = 0`, relever `FLASH_SR`, puis remettre `PG = 0`.

La Flash du L476 se programme par double mot de 64 bits. Les deux écritures forment donc une seule opération, déclenchée par l'écriture du second mot. Le programme tournant en bank 1, il continue de s'exécuter pendant que la bank 2 est programmée.

Résultat : `sr` = `0x0`, `0x08080000` = `0xDEADBEEF`, `0x08080004` = `0xAABBCCDD`. Dans le Memory Browser, on lit en little-endian `EF BE AD DE DD CC BB AA`.

Expression	Type	Value
(x)= val_flash	volatile uint32_t	0x20018000 (Hex)
(x)= val_ram	volatile uint32_t	0x4c4b400 (Hex)
(x)= readback	volatile uint32_t	0xffffffff (Hex)
(x)= flash20000	volatile uint32_t	0xffffffff (Hex)
(x)= sr	volatile uint32_t	0xa8 (Hex)
(x)= flash80004	volatile uint32_t	0x0 (Hex)
(x)= flash80000	volatile uint32_t	0x8003258 (Hex)
(x)= old	volatile uint32_t	0x4c4b400 (Hex)
(x)= FLASH->SR	volatile uint32_t	0x0 (Hex)
(x)= FLASH->CR	volatile uint32_t	0x40000000 (Hex)



Réécriture sans effacement

On tente d'écrire `0x01234567` à la place de `0xAABBCCDD`. La valeur ne change pas et `FLASH→SR` indique une erreur (**PROGERR**, bit 3).

La programmation ne peut faire passer un bit que de 1 à 0. Seul un effacement (par page ou par bank) remet les bits à 1. Avant de programmer, le contrôleur vérifie que le double mot cible est à `0xFFFFFFFFFFFFFFFF`. Ici il contient déjà `0xDEADBEEF_AABBCCDD`, donc l'écriture est refusée.

Expression	Type	Value
(x)= val_flash	volatile uint32_t	0x20018000 (Hex)
(x)= val_ram	volatile uint32_t	0x4c4b400 (Hex)
(x)= readback	volatile uint32_t	0xffffffff (Hex)
(x)= flash20000	volatile uint32_t	0xffffffff (Hex)
(x)= sr	volatile uint32_t	0xa8 (Hex)
(x)= flash80004	volatile uint32_t	0x1234567 (Hex)
(x)= flash80000	volatile uint32_t	0xdeadbeef (Hex)
(x)= old	volatile uint32_t	0x4c4b400 (Hex)
(x)= FLASH->SR	volatile uint32_t	0x0 (Hex)
(x)= FLASH->CR	volatile uint32_t	0x40000000 (Hex)

Remarque observée en pratique

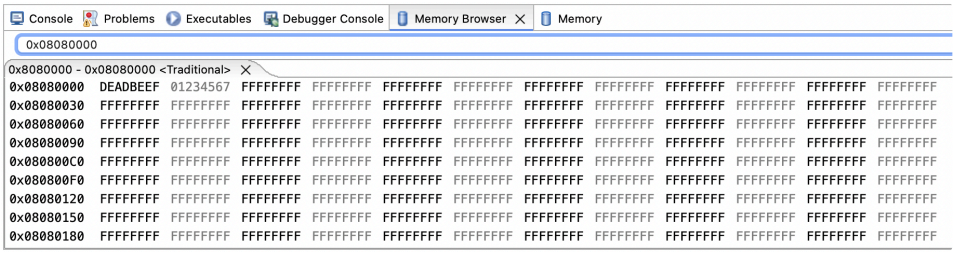
Lors d'un premier essai, sans effacement préalable, la première écriture a été refusée (`sr` = `0xA8`, qui contient PROGERR). Les données écrites lors d'une session de debug précédente étaient encore en bank 2 (`0xDEADBEEF` et `0x01234567`). Cela confirme que la Flash est

non volatile, ses données survivent au reset et à la reprogrammation du bank 1. Le programme final efface donc la bank 2 avant la première écriture.

Effacement de la bank 2

Procédure « Read from bank 1 while page erasing in bank 2 » : `MER2 = 1` dans `FLASH_CR` (effacement total de la bank 2), puis `STRT = 1`, attente de `BSY = 0`, et remise à 0 de `MER2`. Résultat : `sr` = `0x0`, et `0x08080000` et `0x08080004` valent `0xFFFFFFFF`.

Expression	Type	Value
(x)= val_flash	volatile uint32_t	0x20018000 (Hex)
(x)= val_ram	volatile uint32_t	0x4c4b400 (Hex)
(x)= readback	volatile uint32_t	0xffffffff (Hex)
(x)= flash20000	volatile uint32_t	0xffffffff (Hex)
(x)= sr	volatile uint32_t	0x0 (Hex)
(x)= flash80004	volatile uint32_t	0x1234567 (Hex)
(x)= flash80000	volatile uint32_t	0xdeadbeef (Hex)
(x)= old	volatile uint32_t	0x4c4b400 (Hex)
(x)= FLASH->SR	volatile uint32_t	0x0 (Hex)
(x)= FLASH->CR	volatile uint32_t	0x40000000 (Hex)



Nouvelle écriture

Après l'effacement, on réécrit `0xDEADBEEF` et `0x01234567` : `sr` = `0x0`, et les valeurs sont bien lues en mémoire. Le Flash est donc à nouveau programmable après un effacement.

Expression	Type	Value
(x)= val_flash	volatile uint32_t	0x20018000 (Hex)
(x)= val_ram	volatile uint32_t	0x4c4b400 (Hex)
(x)= readback	volatile uint32_t	0xffffffff (Hex)
(x)= flash20000	volatile uint32_t	0xffffffff (Hex)
(x)= sr	volatile uint32_t	0x0 (Hex)
(x)= flash80004	volatile uint32_t	0xaabbccdd (Hex)
(x)= flash80000	volatile uint32_t	0xdeadbeef (Hex)
(x)= old	volatile uint32_t	0x4c4b400 (Hex)
(x)= FLASH->SR	volatile uint32_t	0x0 (Hex)
(x)= FLASH->CR	volatile uint32_t	0x40000000 (Hex)

Synthèse

Opération	Résultat
Lecture Flash / RAM	possible, sans condition
Écriture RAM	libre, mot par mot
Écriture Flash verrouillée	refusée (PGSERR)
Écriture Flash après déverrouillage et effacement	possible, par double mot de 64 bits
Réécriture sans effacement	refusée (PROGERR)
Effacement	remet tous les bits à 1 (`0xFF`)

Clignotement de LED et patch binaire

```c
/* USER CODE BEGIN WHILE */
while (1)
{
    HAL_GPIO_TogglePin(GPIOB, GPIO_PIN_0);
    HAL_Delay(500);
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
}
/* USER CODE END 3 */
```

Le désassemblage

```
080004c4:   movs    r1, #1
080004c6:   ldr     r0, [pc, #16]   @ (0x80004d8 <main+40>)
080004c8:   bl      0x8000ea0 <HAL_GPIO_TogglePin>
```

Dans la vue Disassembly on retrouve l'appel à `HAL_GPIO_TogglePin(GPIOB, GPIO_PIN_0)` :

- `movs r1, #1` charge le masque de la broche (`GPIO_PIN_0 = 0x0001`) dans `r1`, le deuxième argument de la fonction.
- `ldr r0, [pc, #16]` charge l'adresse de `GPIOB` dans `r0`, le premier argument.
- `bl HAL_GPIO_TogglePin` appelle la fonction, qui inverse les bits de `ODR` sélectionnés par le masque.

Le masque n'est donc pas dans la fonction `HAL_GPIO_TogglePin` : il est chargé juste avant l'appel, par `movs r1, #1`. Si on le change, la fonction inversera d'autres broches.

Le patch

Pour faire clignoter `PB0` et `PB1` ensemble, il faut un masque `0x03` (bits 0 et 1) à la place de `0x01`. Il suffit donc de transformer `movs r1, #1` en `movs r1, #3`.

	Instruction	Opcode (16 bits)	Octets en mémoire
Avant	`movs r1, #1`	`0x2101`	`01 21`
Après	`movs r1, #3`	`0x2103`	`03 21`

La manipulation dans STM32CubeProgrammer

La Flash commence à `0x08000000` et l'instruction est à `0x080004C4`. On l'affiche dans l'onglet *Device memory* en lisant 16 octets à partir de `0x080004C0` :

Adresse	Avant	Après
`0x080004C0`	`48042101`	`48042103`

Le mot est affiché sur 32 bits en little-endian : les octets en mémoire sont `01 21 04 48`, c'est-à-dire `movs r1, #1` suivi du début du `ldr r0, [pc, #16]` (`04 48`). On modifie uniquement le dernier chiffre. Les autres mots de la zone ne changent pas.

Étapes :

1. Connecter la carte dans CubeProgrammer et lire `0x080004C0` (taille `0x10`).
2. Double-cliquer sur le mot `48042101`, saisir `48042103` et valider avec Entrée.
3. Relire la zone pour vérifier que la valeur est bien `48042103`.
4. Cliquer sur *Disconnect*, sinon CubeIDE ne peut pas utiliser le ST-LINK.
5. Faire un reset avec le bouton noir de la Nucleo.

Résultat

Après le reset, les deux LED clignotent ensemble à 1 Hz, alors que le code source n'a pas changé.

Pour le vérifier, on lance le debug dans CubeIDE avec l'option *Download* à `false` dans l'onglet *Startup*. Sans cela, CubeIDE reprogramme la Flash avec le binaire d'origine et efface le patch. Dans la vue Disassembly on lit alors :

```Unkown
080004c4:   movs    r1, #3
```

alors que la ligne C affiche toujours `HAL_GPIO_TogglePin(GPIOB, GPIO_PIN_0)`.

	Code source C	Binaire en Flash	LED
Avant le patch	`GPIO_PIN_0`	`movs r1, #1`	LED1 seule
Après le patch	`GPIO_PIN_0`	`movs r1, #3`	LED1 et LED2

Protection en écriture mémoire (SRAM2 et Flash)

Élément	Valeur
SRAM2	`0x10000000` (32 pages de 1 Ko)
2e page	`0x10000400`
`SYSCFG_SWPR`	`0x40010020`

```C
/* USER CODE BEGIN 2 */
volatile uint32_t r1, r2, r3;
#define TEST_FAULT 0 /* 0: On peut écrire, 1 on peut pas écrire */

__HAL_RCC_SYSCFG_CLK_ENABLE();

*(volatile uint32_t *)0x10000000 = 0xDEADBEEF;
r1 = *(volatile uint32_t *)0x10000000;

SYSCFG->SWPR = 0x1;

#if TEST_FAULT
*(volatile uint32_t *)0x10000000 = 0x11223344; /* écriture refusée */
r2 = *(volatile uint32_t *)0x10000000; /* relecture */
#else
*(volatile uint32_t *)0x10000400 = 0xAABBCCDD; /* 2e page : non protégée */
r3 = *(volatile uint32_t *)0x10000400; /* 0xAABBCCDD, pas de reset */
#endif

/* USER CODE END 2 */
```

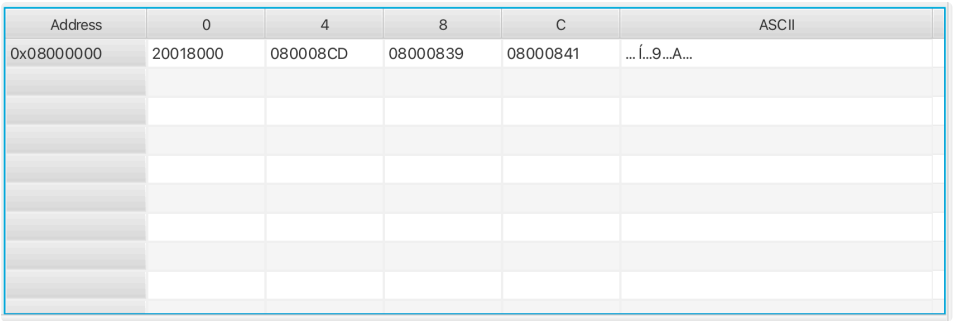
Pour ce programme on écrit d'abord `0xDEADBEEF` au début de la SRAM2 (`0x10000000`) et on le relit dans `r1`. Ensuite on active l'horloge de SYSCFG avec `__HAL_RCC_SYSCFG_CLK_ENABLE()` (sans cela, `SYSCFG->SWPR` n'est pas accessible), puis `SYSCFG->SWPR = 0x1` protège la première page. Le programme existe en deux versions selon `TEST_FAULT` :

- `TEST_FAULT = 1` : on écrit `0x11223344` dans la page protégée, puis on relit.
- `TEST_FAULT = 0` : on écrit `0xAABBCCDD` au début de la 2e page (`0x10000400`), qui n'est pas protégée, sans toucher à la première.
- `TEST_FAULT = 1` (écriture dans la page 0 protégée) : l'écriture est refusée et le programme finit dans une boucle infinie de crash. L'écriture lève une faute, le processeur entre dans `HardFault_Handler`, qui lance le reset applicatif (`SCB->AIRCR = 0x05FA0004`). Le programme redémarre, réécrit `0xDEADBEEF`, reprotège la page 0, retente l'écriture interdite, et ainsi de suite. `SWPR` repasse à 0 à chaque reset, mais le programme le remet à 1 à chaque tour.
- `TEST_FAULT = 0` (écriture dans la page 1) : l'écriture fonctionne, `r3` vaut `0xAABBCCDD` et il n'y a aucun problème ni reset. La protection ne concerne que les pages dont le bit de `SWPR` est à 1.

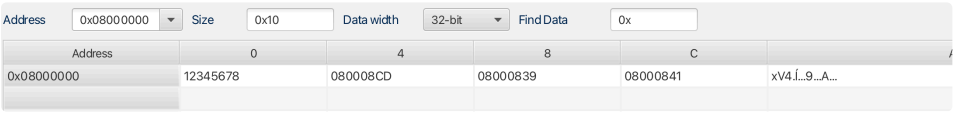
Cas	Adresse écrite	Page protégée ?	Résultat
`TEST_FAULT = 1`	`0x10000000`	oui (bit 0 de `SWPR`)	HardFault, reset, boucle infinie de crash
`TEST_FAULT = 0`	`0x10000400`	non	écriture réussie, `r3` = `0xAABBCCDD`

Les bits de `SYSCFG_SWPR` sont en écriture seule vers 1 : le logiciel peut les mettre à 1 mais pas les remettre à 0. Seul un reset système désactive la protection. Un code malveillant ne peut donc pas la lever une fois qu'elle est activée.

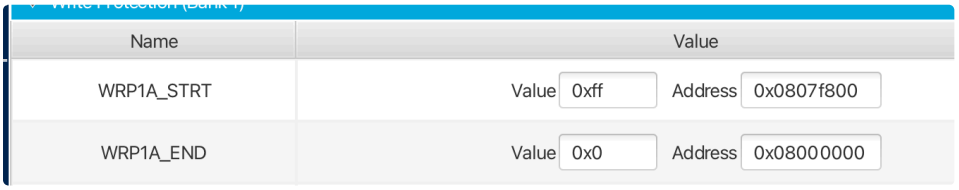
Dans STM32CubeProgrammer, on lit 16 octets à partir de `0x08000000` :



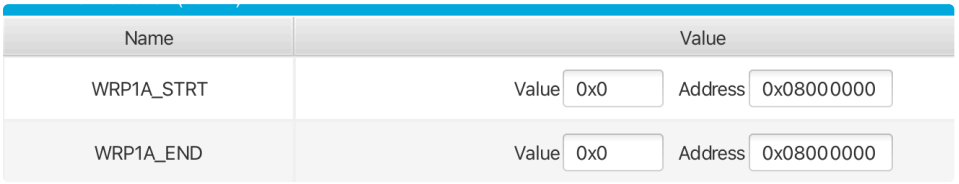
On remplace le premier mot par `12345678` et on relit : la valeur est bien modifiée, les autres mots ne changent pas. Sans protection, la Flash est donc modifiable depuis l'outil de programmation.



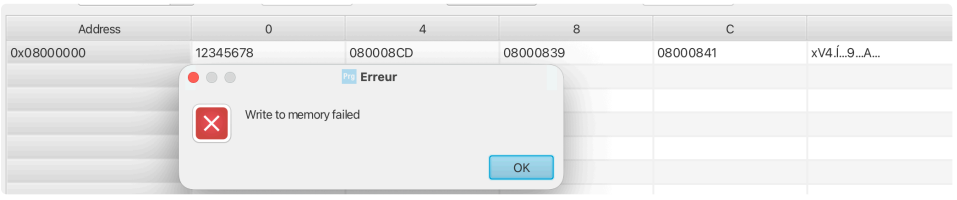
Ici `STRT` (`0xFF`) est supérieur à `END` (`0x0`), donc la zone est vide : aucune page n'est protégée.



On met `WRP1A_STRT = 0x0` et `WRP1A_END = 0x0`, puis *Apply*. Comme `STRT ⩽ END`, la zone protégée va de la page 0 à la page 0, c'est-à-dire `0x08000000` à `0x080007FF`.



On tente de réécrire le premier mot. CubeProgrammer affiche l'erreur « **Write to memory failed** » et la relecture montre que le mot vaut toujours `12345678` : la page 0 est protégée en écriture et la modification est refusée.



Mémoire	Protection	Réversible par le logiciel ?
SRAM2	`SYSCFG_SWPR`, pages de 1 Ko	Non, seulement par reset
Flash	option bytes `WRPxy`, pages de 2 Ko	Non, via un outil externe (CubeProgrammer)

Protection en lecture de la mémoire (RDP)

```c
/* USER CODE BEGIN PV */
volatile uint32_t cpt = 0;
/* USER CODE END PV */

/* USER CODE BEGIN 2 */
*((uint32_t *)0x20000020) = 0xDEADBEEF;
/* USER CODE END 2 */

/* USER CODE BEGIN WHILE */
while (1) {
    HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);
    HAL_Delay(1000);
    cpt++;
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
}
```

Pour ce programme la LED (`PA5`) change d'état toutes les secondes et `cpt` est incrémenté à chaque tour. En debug, `&cpt` donne `0x200000b0`, en SRAM1. On écrit aussi `0xDEADBEEF` à `0x20000020` pour avoir une valeur repérable en SRAM. La RDP (Read Out Protection) protège la Flash contre la copie du code, le reverse engineering et l'accès par les outils de développement. Elle a trois niveaux :

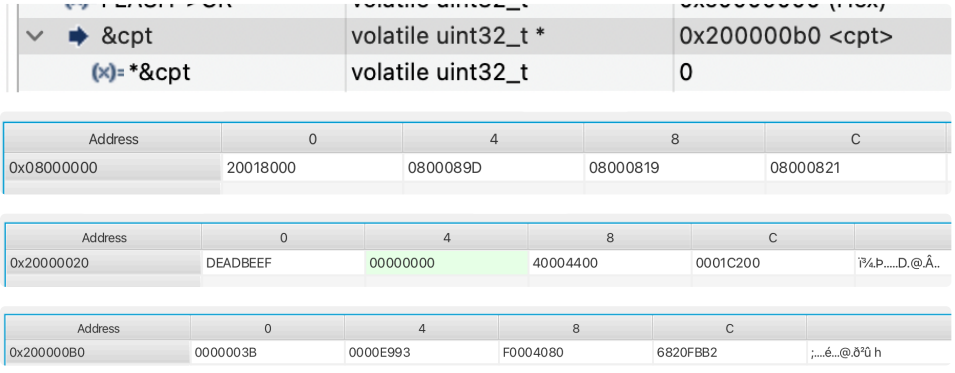
- **Niveau 0** (`RDP = 0xAA`) : aucune protection, c'est le niveau par défaut.
- **Niveau 1** (toute valeur différente de `0xAA` et `0xCC`, ici `0xBB`) : si un accès débogueur est détecté, la lecture de la Flash est bloquée et un HardFault est généré.
- **Niveau 2** (`RDP = 0xCC`) : irréversible, la Flash devient une ROM. Il n'a **pas été testé** : il rend la carte inutilisable.

Niveau 0

Dans STM32CubeProgrammer (onglet *Device memory*), on lit la Flash et la SRAM :

Adresse	Contenu
`0x08000000`	`20018000` `0800089D` `08000819` `08000821`
`0x20000020`	`DEADBEEF` `00000000` `40004400` `0001C200`
`0x200000B0`	`0000003B` (`cpt` = 59)

Tout est lisible. Le premier mot de la Flash est la valeur initiale de `SP` (`_estack`), le deuxième l'adresse de `Reset_Handler`. En SRAM, on retrouve bien `0xDEADBEEF` à `0x20000020`, et `cpt` vaut `0x3B`.



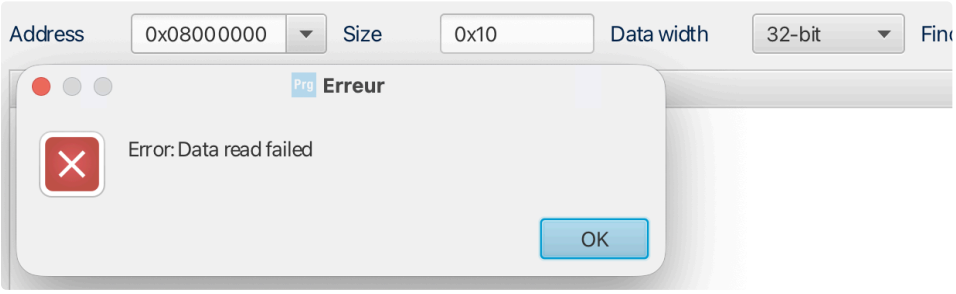
Niveau 1

On passe `RDP` à `0xBB` dans l'onglet *OB* , puis on fait un power cycle (débrancher puis rebrancher l'USB). Résultat :

Zone	Résultat
Flash (`0x08000000`)	« Error: Data read failed » : lecture refusée
SRAM (`0x20000020`, `0x200000B0`)	lecture possible

Zone	Résultat
LED	ne clignote plus

La Flash est illisible depuis l'extérieur, ce qui empêche de copier le firmware. En revanche la SRAM reste lisible : c'est le cas signalé dans le sujet, problématique pour la sécurité, puisque les variables (et donc d'éventuelles données sensibles) restent accessibles. Une solution est de ne stocker ces données que dans les registres du processeur. La LED ne clignote plus car, au niveau 1, un accès débogueur détecté (ici le ST-LINK connecté) bloque l'accès à la Flash et génère un HardFault : le processeur ne peut plus exécuter le programme.



Régression du niveau 1 vers le niveau 0

On remet `RDP = 0xAA`. Cette régression déclenche un effacement total de la Flash : la lecture de `0x08000000` donne `0xFFFFFFFF`. La valeur stockée dans la Flash est donc `0xFFFFFFFF` (cellules effacées, tous les bits à 1) et le programme n'existe plus : la LED ne clignote plus tant qu'on ne reflashe pas le firmware.

Le firmware est détruit, ce qui empêche de le récupérer en repassant au niveau 0 : on protège le code, mais l'ouverture de l'accès coûte l'effacement de la Flash.



Protection propriétaire de code en exécution seule (PCROP)

La protection PCROP (*Proprietary Code Read-Out Protection*) permet d'isoler une partie de la mémoire Flash pour qu'elle soit exécutable uniquement (*Execute-Only*). Toute tentative de lecture (par le processeur via un pointeur/donnée ou par un débogueur externe JTAG/SWD) provoque une erreur de bus (*BusFault*), tandis que le processeur conserve le droit d'y chercher des instructions pour les exécuter. Cette fonctionnalité répond typiquement au cas industriel où une entreprise A fournit une brique logicielle ou une propriété intellectuelle à une entreprise B sans que cette dernière ne puisse rétro-ingénier ou dumper le code machine.

Pour ce TP, l'IP à protéger est la fonction `secret_led_blinking` définie dans un fichier dédié `led_blinking.c` :

```c
#include "main.h"

#define LED_PORT_GPIO_ADDRESS *((uint32_t *) 0x48000014) // GPIOA_ODR
#define GPIO_PIN_LED (1 << 5)                            // PA5 (LED verte)

void secret_led_blinking(void) {
    LED_PORT_GPIO_ADDRESS ^= GPIO_PIN_LED;
}
```

Dans la fonction `main` du projet, cette fonction est appelée périodiquement :

```c
/* USER CODE BEGIN 3 */
secret_led_blinking();
HAL_Delay(500);
/* USER CODE END 3 */
```

État initial sans protection

Au départ, la fonction est compilée dans la section standard `.text` en Flash classique. Dans la vue Disassembly, l'appel depuis `main` se fait vers l'adresse standard `0x080004f4` :

```
Unkown
 bl      0x80004f4 <secret_led_blinking>
104             HAL_Delay(500);
```

À ce stade, n'importe qui accédant à la carte avec un ST-LINK peut désassembler ou dumper le code binaire de `secret_led_blinking`.

Déplacement de la fonction dans le Linker Script

Afin de pouvoir appliquer la protection matérielle sur une zone spécifique de la mémoire, on segmente la Flash dans le fichier `STM32L476RGTX_FLASH.ld`. La Flash étant découpée en pages de 2 Ko, on réserve une région `PCROP` de 16 Ko débutant à `0x08008000`, et on réduit la Flash non sécurisée à 16 Ko :

```
Unkown
MEMORY
{
  RAM   (xrw) : ORIGIN = 0x20000000, LENGTH = 96K
  RAM2  (xrw) : ORIGIN = 0x10000000, LENGTH = 32K
  FLASH (rx)  : ORIGIN = 0x08000000, LENGTH = 16K
  PCROP (rx)  : ORIGIN = 0x08008000, LENGTH = 16K
}
```

Puis, dans la section `SECTIONS`, on crée la section `.PCROPed` qui y dirige spécifiquement le code de `led_blinking.o` :

```
Unkown
.PCROPed :
{
  . = ALIGN(4);
  *led_blinking.o (.text .text*)
  . = ALIGN(4);
} > PCROP
```

Configuration du compilateur (-mslow-flash-data)

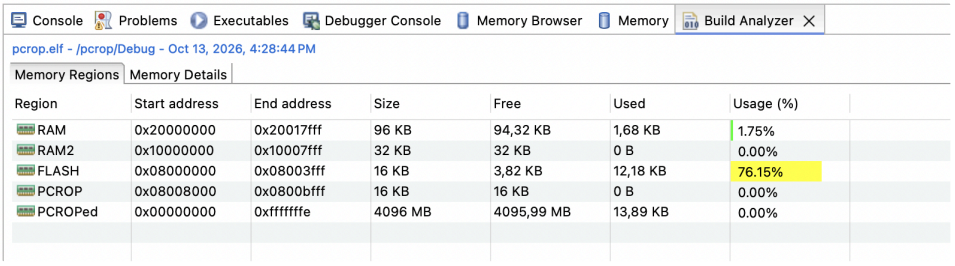
Comme la zone PCROP est en *Execute-Only*, tout accès en lecture de donnée est rigoureusement interdit. Or, par défaut, le compilateur GCC place souvent les constantes ou les adresses de périphériques dans un *literal pool* en Flash et les lit via des instructions `ldr rX, [pc, #offset]`. Dans une zone PCROP, cette lecture déclencherait immédiatement une faute.

Pour éviter cela, on modifie les options de compilation spécifiques au fichier `led_blinking.c` (clic droit sur le fichier -> *Properties* -> *C/C++ Build* -> *Settings* -> *Tool Settings* -> *MCU GCC Compiler*) :

1. Optimisation : `-O0` (*None*).
2. Option cochée : « **Assume loading data from flash is slower than fetching instruction (-mslow-flash-data)** ».

Cette directive force GCC à générer les valeurs immédiates et les adresses directement sous forme d'instructions arithmétiques séquentielles (`movw`, `movt`, `orr`), garantissant qu'aucune donnée n'est lue en Flash.

Dans le Build Analyzer, on vérifie que la zone `PCROP` est bien créée à `0x08008000` :



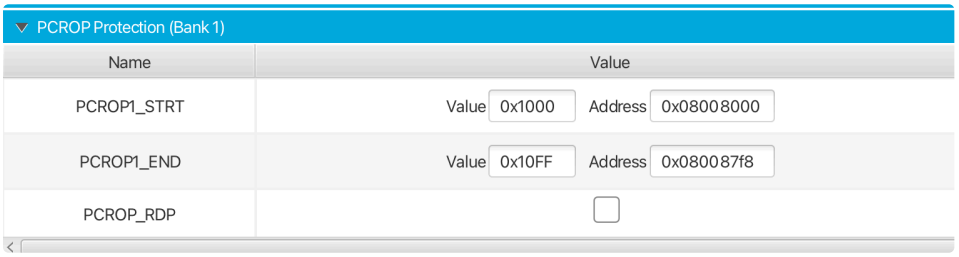
Et dans la vue Disassembly, on constate que le saut `bl` cible désormais l'adresse `0x08008000` :

```
Unkown
08000508:   bl      0x8008000 <secret_led_blinking>
104             HAL_Delay(500);
```

Activation matérielle de la protection PCROP

La protection s'active au niveau des Option Bytes dans STM32CubeProgrammer :

1. Dans l'onglet *OB* -> section *PCROP Protection (Bank 1)*.
2. On configure la plage d'adresses à protéger :
   - `PCROP1_STRT = 0x1000` (correspondant à l'adresse de départ `0x08008000`).
   - `PCROP1_END = 0x10FF` (adresse de fin `0x080087F8`).
3. On clique sur *Apply*.



Observation en mode Debug

Une fois PCROP activé, le contrôleur Flash bloque toute lecture de cette zone par le ST-LINK.
Pour relancer une session de débogage sans écraser ni bloquer la mémoire :

- Dans la configuration de débogage (*Debug Configurations* -> onglet *Startup*), il est indispensable de passer l'option **Download à** `false`. Si l'option reste active, CubeIDE tente d'effacer et reprogrammer la Flash protégée, ce qui échoue.

Au redémarrage de la cible :

- **Comportement physique** : la LED verte continue de clignoter normalement à 1 Hz. Le processeur lit et exécute les instructions sans aucune entrave depuis la zone PCROP.
- **Comportement du débogueur** : lorsque l'on inspecte la fonction `secret_led_blinking` à l'adresse `0x08008000` dans la fenêtre Disassembly ou Memory Browser, la mémoire apparaît masquée (lecture retournant `0xFFFF`). Le débogueur est incapable de restituer le code assembleur d'origine. La propriété intellectuelle est ainsi préservée contre toute tentative d'analyse.

Désactivation de la protection PCROP

La protection PCROP ne peut pas être désactivée par une simple réécriture des registres de configuration. Pour la supprimer :

1. Dans STM32CubeProgrammer, cocher l'option `PCROP_RDP`.
2. Régresser le niveau RDP en passant de **Niveau 1 (`0xBB`)** à **Niveau 0 (`0xAA`)**.
3. Cette opération entraîne obligatoirement un **effacement de masse (*Mass Erase*)** de l'ensemble de la Flash.

Ce mécanisme garantit qu'il est impossible de lever la protection pour lire frauduleusement le code, toute tentative de déverrouillage détruit irrémédiablement le binaire protégé.

Isolation et enclave logicielle (Firewall)

Le périphérique Firewall intégré au STM32L476 permet de créer une enclave de sécurité matérielle isolant une portion de code et de données au sein d'une même application. Contrairement à une simple protection globale de la mémoire, le Firewall filtre les transactions directement sur la matrice de bus AHB. Il définit trois segments protégés distincts :

- Un **segment de code** (en Flash).
- Un **segment de données non volatiles** (en Flash).
- Un **segment de données volatiles** (en SRAM).

L'accès à cette enclave est strictement conditionné : toute exécution du code protégé doit obligatoirement débuter par une adresse précise appelée la ***Call Gate*** (`CodeSegmentStartAddress + 4`). Dès lors qu'une instruction valide franchit cette porte, le Firewall passe à l'état

ouvert (*Open*) et autorise l'accès aux segments de données associés. Tout accès non conforme (saut au milieu du code, lecture de données protégées depuis le code externe, interruption non maîtrisée) provoque immédiatement un reset matériel du microcontrôleur.

Le composant ne pouvant pas être configuré graphiquement via CubeMX, l'intégration de son pilote et sa configuration doivent être réalisées manuellement.

Intégration manuelle du pilote HAL Firewall

Les pilotes bas niveau fournis par STMicroelectronics sont intégrés dans l'arborescence du projet :

- `stm32l4xx_hal_firewall.c` dans `Drivers/STM32L4xx_HAL_Driver/Src/`
- `stm32l4xx_hal_firewall.h` dans `Drivers/STM32L4xx_HAL_Driver/Inc/`

Le module est ensuite activé dans le fichier de configuration `Core/Inc/stm32l4xx_hal_conf.h` :

```c
#define HAL_FIREWALL_MODULE_ENABLED
```

```
/*#define HAL_DSI_MODULE_ENABLED   */
#define HAL_FIREWALL_MODULE_ENABLED
/*#define HAL_GFXMMU_MODULE_ENABLED   */
```

Segmentation mémoire dans le Linker Script

Afin d'isoler physiquement les sections protégées, le fichier de liaison `STM32L476RGTX_FLASH.ld` est partitionné. La mémoire Flash non sécurisée est réduite à 1008 Ko, et deux zones dédiées sont déclarées :

```
MEMORY
{
  RAM         (xrw) : ORIGIN = 0x20000000, LENGTH = 96K
  RAM2        (xrw) : ORIGIN = 0x10000000, LENGTH = 32K
  FLASH        (rx) : ORIGIN = 0x08000000, LENGTH = 1008K
  SecureFlash  (rx) : ORIGIN = 0x080FC004, LENGTH = 0x3FFC
  SecureSRAM  (xrw) : ORIGIN = 0x2000FE00, LENGTH = 512
}
```

```
MEMORY
{
  RAM     (xrw)    : ORIGIN = 0x20000000,   LENGTH = 96K
  RAM2     (xrw)    : ORIGIN = 0x10000000,   LENGTH = 32K
  FLASH    (rx)    : ORIGIN = 0x8000000,   LENGTH = 1024K
  SecureFlash (rx)  : ORIGIN = 0x080FC004, LENGTH = 0x3FFC
  SecureSRAM  (xrw) : ORIGIN = 0x2000FE00, LENGTH = 512
}
```

> **Note**
>
> La zone `SecureFlash` débute à l'adresse `0x080FC004`. En effet, l'ouverture matérielle du Firewall s'effectue obligatoirement à l'adresse de base du segment de code incrémentée de 4 octets (`0x080FC000 + 4`). La première fonction de cette section constitue donc la *Call Gate*. Par ailleurs, pour éviter un dépassement de 4 octets au-delà de la fin physique de la Flash (`0x08100000`), la taille allouée est ajustée à `0x3FFC` (soit **16 380 octets**, c'est-à-dire 16 Ko moins 4 octets) au lieu des 16 Ko théoriques (`0x080FC004 + 0x3FFC = 0x08100000`).

Dans le bloc `SECTIONS`, les règles de placement du code et des données protégées sont ajoutées :

```
.mysection :
{
  . = ALIGN(4);
  *(.mysection*)
```

```
  . = ALIGN(4);
} > SecureFlash

.securedata (NOLOAD) :
{
  *(.securedata)
  *(.securedata*)
} > SecureSRAM
```

```
.mysection :
{
  . = ALIGN(4);
  *(.mysection*)
  . = ALIGN(4);
} > SecureFlash

.securedata (NOLOAD) :
{
  *(.securedata)
  *(.securedata*)
} > SecureSRAM
```

La directive `NOLOAD` indique au linker que la section `.securedata` ne doit pas faire l'objet d'un chargement initial en mémoire Flash par le programmateur, s'agissant de variables non initialisées allouées dynamiquement en SRAM.

Implémentation du code sécurisé et configuration

La variable confidentielle `number_calls` est placée dans la section protégée `.securedata` :

```c
/* USER CODE BEGIN PV */
__attribute__((section(".securedata"))) uint32_t number_calls;
/* USER CODE END PV */
```

Les fonctions exécutées dans l'enclave sont définies dans la section `.mysection` :

```c
/* USER CODE BEGIN 0 */
void __attribute__((section(".mysection"))) Protected_function(void);
void __attribute__((section(".mysection"), noinline)) toggleLED(void);

void __attribute__((section(".mysection"))) Protected_function(void)
{
    __HAL_FIREWALL_PREARM_DISABLE();
    toggleLED();
    number_calls++;
    __HAL_FIREWALL_PREARM_ENABLE();
}

void __attribute__((section(".mysection"), noinline)) toggleLED(void)
{
    GPIOA->ODR ^= GPIO_PIN_5;
}
/* USER CODE END 0 */
```

- `Protected_function` est déclarée en premier dans le fichier source afin d'être physiquement positionnée à l'adresse de la *Call Gate* (`0x080FC004`).
- `toggleLED` modifie directement le registre `GPIOA->ODR` sans passer par les pilotes HAL, car tout saut hors de la section protégée pendant que le Firewall est actif déclencherait immédiatement un reset.

- L'attribut `noinline` empêche le compilateur d'intégrer le corps de `toggleLED` directement dans `Protected_function`, conservant la structure d'appel de sous-programme demandée.

L'initialisation et le verrouillage s'effectuent au démarrage dans la routine principale :

```c
/* USER CODE BEGIN 2 */
__HAL_RCC_SYSCFG_CLK_ENABLE();
number_calls = 0;

FIREWALL_InitTypeDef fw_init = {0};
fw_init.CodeSegmentStartAddress = 0x080FC000;
fw_init.CodeSegmentLength = 0x2000;
fw_init.NonVDataSegmentStartAddress = 0x080FE000;
fw_init.NonVDataSegmentLength = 0x200;
fw_init.VDataSegmentStartAddress = 0x2000FE00;
fw_init.VDataSegmentLength = 0x200;
fw_init.VolatileDataExecution = FIREWALL_VOLATILEDATA_NOT_EXECUTABLE;
fw_init.VolatileDataShared = FIREWALL_VOLATILEDATA_NOT_SHARED;

if (HAL_FIREWALL_Config(&fw_init) != HAL_OK) Error_Handler();
HAL_FIREWALL_EnableFirewall();
/* USER CODE END 2 */
```

Remarques relatives à la configuration :

1. L'horloge du périphérique SYSCFG doit impérativement être validée au préalable (`__HAL_RCC_SYSCFG_CLK_ENABLE()`), car le bit d'activation `FWDIS` réside dans `SYSCFG→CFGR1`.
2. La variable `number_calls` est initialisée à zéro avant l'armement du Firewall, puisqu'elle deviendra inaccessible depuis ce contexte une fois la protection verrouillée.
3. Le pilote HAL impose que le segment de données non volatiles ait une taille minimale de `0x100` sous peine de renvoyer `HAL_ERROR`, une plage de `0x200` octets lui est donc assignée afin d'avoir un peu de marge.
4. Une fois armé via `HAL_FIREWALL_EnableFirewall()`, le bit `FWDIS` est verrouillé, aucun mécanisme logiciel ne peut désactiver le Firewall, seul un reset matériel remet l'état à zéro.

Dans la boucle infinie, les interruptions sont masquées pendant l'appel afin de garantir qu'aucune routine ISR ne tente d'interrompre l'enclave :

```c
/* USER CODE BEGIN WHILE */
while (1)
{
    __disable_irq();
    Protected_function();
    //volatile uint32_t x = number_calls;
    //toggleLED();
    __enable_irq();
    HAL_Delay(500);
    /* USER CODE END WHILE */
    /* USER CODE BEGIN 3 */
}
/* USER CODE END 3 */
```

Analyse de l'implantation mémoire (Fichier .map)

L'inspection du fichier `.map` issu de la compilation confirme le respect strict de la topologie mémoire :

Élément	Adresse physique	Propriété / Rôle
`Protected_function`	`0x080FC004`	Porte d'entrée unique (*Call Gate*)
`toggleLED`	`0x080FC058`	Fonction interne à l'enclave
`.mysection`	`0x080FC004` – `0x080FC078`	Segment Flash protégé (taille `0x74` octets)
`number_calls`	`0x2000FE00`	Variable confidentielle en SRAM sécurisée
`FLASH` classique	`0x08000000`	Code non sécurisé, longueur `0xFC000` (1008 Ko)

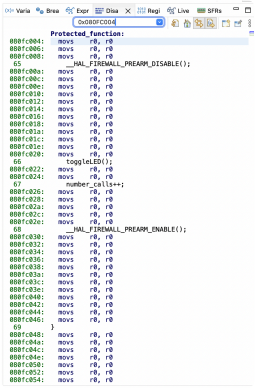
```
3192 .mysection      0x080fc004        0x74
3193                 0x080fc004                        . = ALIGN (0x4)
3194  *(.mysection*)
3195  .mysection     0x080fc004        0x74 ./Core/Src/main.o
3196                 0x080fc004                Protected_function
3197                 0x080fc058                toggleLED
3198                 0x080fc078                        . = ALIGN (0x4)
3199
3200 .securedata     0x2000fe00         0x4
3201  *(.securedata)
3202  .securedata    0x2000fe00         0x4 ./Core/Src/main.o
3203                 0x2000fe00                number_calls
3204  *(.securedata*)
3205
```

Test 1 : Fonctionnement nominal et Pre-Arm

Dans des conditions normales d'exécution, la LED verte (PA5) clignote toutes les 500 ms. L'entrée dans `Protected_function` par la *Call Gate* valide ouvre le Firewall, la LED bascule, le compteur s'incrémente, puis l'instruction `__HAL_FIREWALL_PREARM_ENABLE()` réarme le mécanisme de fermeture automatique à la sortie.

Registre	Valeur relevée	Signification technique
`FIREWALL→CR`	`0x1`	Bit `FPA` à 1 : mode Pre-Arm validé en sortie
`SYSCFG→CFGR1`	`0x7C000000`	Bit `FWDIS` à 0 : Firewall armé et actif
`&number_calls`	`0x2000FE00`	Adresse confirmée dans le segment SRAM protégé

Dans la vue Disassembly, la plage `0x080FC004` – `0x080FC054` apparaît au débogueur sous forme d'instructions `movs r0, r0` (octets nuls), ce qui veut dire que le ST-LINK ne peut pas lire le contenu réel du code machine, bien que le processeur l'exécute parfaitement.



Expression	Type	Value
&number_calls	uint32_t *	0x2000fe00 (Hex)
FIREWALL->CR	volatile uint32_t	0x1 (Hex)
SYSCFG->CFGR1	volatile uint32_t	0x7c000000 (Hex)
Add new expression		

Test 2 : Tentative d'appel direct hors Call Gate

Pour valider l'intransigeance du Firewall, l'appel à `Protected_function()` est remplacé dans `main()` par un appel direct à `toggleLED()`. Bien que `toggleLED` soit physiquement située dans le segment de code protégé (`0x080FC058`), l'instruction de branchement court-circuite la *Call Gate* (`0x080FC004`).

Constat : La LED ne clignote plus et le microcontrôleur redémarre instantanément en boucle, repassant continuellement par `HAL_Init()`. Le contrôleur matériel génère un reset immédiat dès la première instruction lue en dehors du point d'entrée officiel.

Type d'appel	Adresse ciblée	Statut de la porte	Comportement observé
`Protected_function()`	`0x080FC004`	*Call Gate* valide	Exécution normale, la LED clignote
`toggleLED()` direct	`0x080FC058`	Entrée illégale	Violation d'accès → Reset immédiat

Le segment protégé ne tolère donc aucune exécution fractionnée, l'application non sécurisée ne peut interagir avec l'enclave que par l'interface formellement publiée.

```
91   /* Reset of all peripherals, Initializes the Flash interface and the Systick. */
92   HAL_Init();
93
```

Test 3 : Tentative d'accès non autorisé aux données protégées

Deux modes de lecture de la variable confidentielle `number_calls` sont évalués :

1. **Accès logiciel depuis `main()`** : l'instruction `volatile uint32_t x = number_calls;` est décommentée dans la boucle principale, en dehors de l'enclave. L'accès en lecture sur le bus AHB est immédiatement intercepté et déclenche le reset continu du système. La SRAM protégée est donc totalement hermétique au reste de l'application.
2. **Accès matériel par le débogueur** : lors d'une halte du processeur dans `main()`, la vue *Expressions* de STM32CubeIDE parvient à afficher la valeur de `number_calls` (valeur observée : `4`) sans provoquer de reset.

Origine de l'accès à `number_calls`	Canal d'accès	Réaction du système
Code applicatif non sécurisé (`main`)	Bus AHB interne	Violation détectée → Reset matériel
Routine sécurisée (`Protected_function`)	Bus interne (Firewall ouvert)	Lecture et écriture autorisées
Sonde de débogage externe (SWD/JTAG)	Bus de debug DAP	Donnée lisible (pas de reset)

Ce test démontre que le Firewall protège les données contre les menaces logicielles internes, mais qu'il doit être obligatoirement couplé à une protection de niveau RDP 1 ou 2 pour interdire la consultation par une sonde physique.

Expression	Type	Value
&number_calls	uint32_t *	0x2000fe00 (Hex)
FIREWALL->CR	volatile uint32_t	0x1 (Hex)
SYSCFG->CFGR1	volatile uint32_t	0x0 (Hex)
number_calls	uint32_t	4
Add new expression		

Test 4 : Analyse des fuites de données par les registres du cœur

Le Firewall filtre rigoureusement les accès mémoires en Flash et en RAM, mais il n'exerce aucun contrôle sur les registres internes du cœur Cortex-M4 (`R0` à `R12`). Si une fonction protégée manipule des données secrètes dans ces registres sans les assainir avant de rendre la main, l'application non sécurisée peut facilement les récupérer.

Pour mettre en évidence cette vulnérabilité, l'incrémentation en C est remplacée par la séquence assembleur suivante :

```c
__asm__ (
    "ldr r4, =number_calls\n"
    "ldr r5, [r4]\n"
    "add r5, #1\n"
    "str r5, [r4]\n"
    "ldr r6, =0xDEADBEEF");
```

Des points d'arrêt sont positionnés dans la boucle `main()`, juste après la sortie de `Protected_function()`. Les registres `R4`, `R5` et `R6` sont ensuite relevés :

Point d'arrêt	Registre `R4` (Adresse)	Registre `R5` (Valeur secrète)	Registre `R6` (Motif témoin)
1	`0x2000FE00`	`0x2C` (44)	`0xDEADBEEF`
2	`0x2000FE00`	`0x2D` (45)	`0xDEADBEEF`

Point d'arrêt	Registre `R4` (Adresse)	Registre `R5` (Valeur secrète)	Registre `R6` (Motif témoin)
3	`0x2000FE00`	`0x2F` (47)	`0xDEADBEEF`

Name	Value
General Registers	
r0	0x561c (Hex)
r1	0x40011c00 (Hex)
r2	0x1f5 (Hex)
r3	0x1f5 (Hex)
r4	0x2000fe00 (Hex)
r5	0x2c (Hex)
r6	0xdeadbeef (Hex)

Name	Value	Descriptic
General Registers		General P
r0	0x561c (Hex)	
r1	0x40011c00 (Hex)	
r2	0x40011c00 (Hex)	
r3	0x1 (Hex)	
r4	0x2000fe00 (Hex)	
r5	0x2d (Hex)	
r6	0xdeadbeef (Hex)	
r7	0x20017fd0 (Hex)	

Name	Value	D
General Registers		G
r0	0x5a06 (Hex)	
r1	0x40011c00 (Hex)	
r2	0x40011c00 (Hex)	
r3	0x1 (Hex)	
r4	0x2000fe00 (Hex)	
r5	0x2f (Hex)	
r6	0xdeadbeef (Hex)	

Constat de la fuite d'information

Bien que le code non sécurisé n'ait aucun droit d'accès direct à la variable `number_calls` en SRAM, il lui suffit de lire les registres à la sortie du sous-programme pour récupérer :

- L'adresse physique de la variable protégée dans `R4` (`0x2000FE00`).
- La valeur du compteur dans `R5` (`44`, `45`, `47`).
- La constante arbitraire dans `R6` (`0xDEADBEEF`).

L'isolation mémoire offerte par le Firewall est donc compromise par la persistance de l'état des registres.

Recommandations et contre-mesures

Afin de combler cette faille architecturale, plusieurs bonnes pratiques doivent être instaurées :

1. **Sauvegarde et restauration conforme (AAPCS)** : Sauvegarder systématiquement les registres non volatils (`R4` à `R11`) sur la pile sécurisée via `push` / `pop` dès l'entrée dans l'enclave, assurant que l'appelant retrouve son propre contexte.
2. **Effacement systématique des registres de travail** : Réinitialiser à zéro les registres volatils (`R0` – `R3`, `R12`) avant le retour d'instruction (`bx lr`), afin qu'aucune donnée intermédiaire ne subsiste.

Il existe probablement d'autre bonnes pratiques mais je n'ai pas eu le temps de les identifier.

Synthèse et bilan de sécurité

Vecteur d'accès	Comportement constaté	Niveau de robustesse
Appel nominal via *Call Gate* (`0x080FC004`)	Exécution normale du code protégé	Autorisé
Appel direct d'une fonction interne (`toggleLED`)	Détection d'intrusion → Reset immédiat	Hermétique
Lecture/écriture de la SRAM sécurisée hors enclave	Transaction interceptée → Reset immédiat	Hermétique
Inspection de la mémoire par sonde SWD	Lecture possible sans blocage	Vulnérable (nécessite RDP)
Fuite de données résiduelles dans les registres	Contenu de `R4`, `R5`, `R6` exploitable	Vulnérable (nécessite assainissement)
Désactivation logicielle du Firewall	Impossible sans un reset matériel complet	Robuste

Le Firewall matériel offre une isolation logicielle efficace et sans surcoût d'émulation pour compartimenter du code critique. Cependant, son étanchéité globale dépend impérativement d'un assainissement rigoureux des registres du processeur à la sortie de l'enclave, ainsi que du verrouillage physique du débogage par le mécanisme RDP.